

Skill registry — 2026-05-22

A register of every custom skill I've written for Claude Code (Anthropic's coding assistant CLI), and whether it has measured data behind its claims yet. A "skill" here is a reusable slash command — a named recipe Claude Code runs when I type `/audit` or similar. 4 repos, 34 skills, 13 of them A/B-tested for token savings so far.

PROJECTED ANNUAL SAVINGS — MEASURED VS MODELED

2.71M 8% of total · real A/B receipts

Measured savings (A/B-tested skills only — 13 of 33 skills A/B-tested so far)



32.7M 3× cost heuristic, no A/B yet

Modeled savings (skills still without measured savings data)



This document is a register of skills that have measured data behind them, not a usage receipt. The measured bar above is the only number on this page derived from real A/B runs — the modeled bar is what the 3× heuristic claims is being saved on the rows that haven't been calibrated yet. Same bar scale, no zoom trickery.

Context: the entire custom-skill portfolio used 4.81M tokens in the last 90 days (sum of transcript-measured rows). `/review` (built-in) ate 14.7M in the same window — ~33× the portfolio's own footprint, shown for scale. See the built-ins table below for the row.

This document carries two kinds of "measured" data, and they're different. **Cost-per-use measurement** (the green chip on a row) comes from real Claude Code transcripts — I went back through my own sessions and counted what each skill actually spent when I invoked it. **Savings-per-use measurement** (the bold green / orange number in the Saved column) comes from A/B tests — same task solved with and without the skill, in fresh sandboxed worktrees, both arms billed honestly. Rows tagged ☐ **ESTIMATE** have neither yet. Every measured estimate I had has turned out to be off, usually by 5× to 100×. The

interesting part isn't that I was wrong. The interesting part is that you can see exactly how wrong, on page 2.

Per-repo summary

REPO	SKILLS	MEASURED	TOKENS USED (90D)	SAVED / YR*
AudiobookMaker	8	2	202K	~1.82M
Spacepotatis	12	4	930K	~2.42M
claude-skills	10	6	3.11M	~26.1M
mikkonumminen.dev	4	2	561K	~4.99M

* Saved is mixed — A/B-measured where the “measured” subline appears, modeled (3× heuristic) for the rest. Italicised values are majority-estimate on the cost side; upright values are majority-measured on the cost side. See the method page for both models.

Claude Code built-ins (reference)

Built-in slash commands. Cost is measured the same way as the custom-skill rows. No SKILL.md exists for these, so they have no procedure to A/B-test against — the calibration column reads “not A/B-tested” and the savings column reads “—” rather than zero. Excluded from every total in the document. Shown so the reader has a scale anchor when reading the custom-skill numbers.

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
/review Claude Code built-in PR code review	✓MEASURED last 2026-05-21	1.05M tokens / use (measured)	14 in 90d ~57/yr projected	not A/B-tested	—

Calibration honesty — where my guesses landed

Here are the rows where I had a guess before I had data. The “How wrong” column is the measurement divided by the guess. Green means I landed within $\pm 10\%$. Orange means I was off by 5 $\times$ or more in either direction. 5 of 12 rows are orange. The fix is not to write better guesses next time. The fix is to keep measuring.

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
CLAUDE-SKILLS mikko-help	7K	644K	99 $\times$ under
SPACEPOTATIS equipment	4K	401K	93 $\times$ under
SPACEPOTATIS new-story	5K	56K	10 $\times$ under
CLAUDE-SKILLS mikko-install	4K	35K	8.8 $\times$ under
CLAUDE-SKILLS skill-usage	4K	25K	6.2 $\times$ under
AUDIOBOOKMAKER copyright-scan	3K	14K	4.6 $\times$ under
SPACEPOTATIS content-audit	15K	53K	3.5 $\times$ under
CLAUDE-SKILLS ai-codegen-smell-audit	75K	199K	2.7 $\times$ under
SPACEPOTATIS save-roundtrip-audit	12K	19K	1.5 $\times$ under
CLAUDE-SKILLS audit	425K	553K	1.3 $\times$ under
MIKKONUMMINEN.DEV sync-readmes	141K	119K	1.2 $\times$ over

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
MIKKONUMMINEN.DEV skill-registry	130K	148K	1.1x under

Per-repo skill tables

One row per skill. Measured rows have a green wash; estimated rows are white. **Saved / yr** carries two kinds of numbers: **bold green tagged *measured*** means a real A/B run (arm A vs arm B) gave us the savings figure; italic gray tagged *modeled* means we're still using the 3× heuristic. Orange numbers are real measurements showing the skill cost MORE per use than the unstructured baseline. The method page describes both.

AudiobookMaker						8 skills · 2 measured · github.com/MikkoNumminen/AudiobookMaker
SKILL	STATUS	COST / USE		RUNS	CALIBRATION	SAVED / YR
ci-failure-triage When CI fails on a release tag or PR, walk the failure modes in the right order against a recipe library built from the project's fix(ci) commit history.	☐ ESTIMATE	1K tokens / use (est.)	89 / yr (est.)	<i>n/a</i>		<i>~178K/yr</i> modeled (from est.)
copyright-scan Scan a git diff (staged by default, or a named revision range / file set) for accidental third-party copyright leaks before they land on origin.	☒ MEASURED last 2026-05-12	14K tokens / use (measured)	1 in 90d ~4/yr projected	4.6× under guess was 3K/use		~111K/yr modeled
pronunciation-corpus-add Append a Finnish pronunciation failure (a word the Chatterbox Grandmom voice mispronounces) to the project's pronunciation corpus at docs...	☐ ESTIMATE	—	—	<i>n/a</i>		—
release-bundle-audit Audit the AudiobookMaker PyInstaller release bundle for unused dependencies, dead-code data files, and accidental ML-stack pollution; then propose spec-only fixes on a `chore/release-bundle-size` branch.	☐ ESTIMATE	3K tokens / use (est.)	4 / yr (est.)	<i>n/a</i>		<i>~24K/yr</i> modeled (from est.)
release-cut Cut a new AudiobookMaker release (bump APP_VERSION, tag vX.Y.Z, push, verify CI lands SHA-256 in release notes AND sidecar). Use this ski...	☒ MEASURED last 2026-05-12	188K tokens / use (measured)	1 in 90d ~4/yr projected	<i>no prior number</i>		~1.51M/yr modeled

SKILL	STATUS	COST / USE		RUNS	CALIBRATION	SAVED / YR
<u>voice-pack-finnish</u> Build a Finnish voice pack from a source audio file end-to-end — chunked analyze with ECAPA diarization, transcript validation per speaker, branching to LoRA training (long sources) or few-shot ref-clip packaging (short sources), and ear-check by synth.	☐ ESTIMATE	7K tokens / use (est.)	—	<i>n/a</i>	—	
<u>work-session</u> Start, pause, or finish a TODO.md work session as one of the 4 permanent Claude sessions in AudiobookMaker. Use this whenever the user sa...	☐ ESTIMATE	2K tokens / use (est.)	—	<i>n/a</i>	—	
<u>worktree-launch</u> Start a new parallel Claude session safely.	☐ ESTIMATE	800 tokens / use (est.)	—	<i>n/a</i>	—	

Spacepotatis

12 skills · 4 measured · github.com/MikkoNumminen/Spacepotatis

SKILL	STATUS	COST / USE		RUNS	CALIBRATION	SAVED / YR
<u>balance-review</u> Diff uncommitted changes to game data (weapons, enemies, waves, missions, perks, augments, loot pools, solar systems) and report DPS, TTK, energy-cost-per-DPS, augment-folded effective DPS, loot-pool roster shifts, and drop-rate deltas vs HEAD.	☐ ESTIMATE	14K tokens / use (est.)	50 / yr (est.)	<i>n/a</i>	~656K/yr measured	
<u>content-audit</u> Pre-commit content invariants check — orphan refs (enemy / weapon / sprite / pod / loot-pool / mission system / story trigger), missing sprite generators, perk drop-weight sanity, mission prereq DAG, story integrity (voice + music files, trigger refs), storyTriggers helper coverage.	☒ MEASURED last 2026-05-03	53K tokens / use (measured)	1 in 90d ~4/yr projected	3.5× under guess was 15K/use	~89K/yr measured	
<u>equipment</u> Create, modify, or remove a weapon or piece of equipment (augment / reactor / shield / armor) — including visual changes to how it appears in combat or in the loadout UI.	☒ MEASURED last 2026-05-03	401K tokens / use (measured)	2 in 90d ~8/yr projected	93× under guess was 4K/use	–37K/yr measured · costs more	

SKILL	STATUS	COST / USE		RUNS	CALIBRATION	SAVED / YR
<u>modular-architecture-audit</u> Orchestrates the multi-phase modular-architecture audit + refactor.	○ ESTIMATE	5K tokens / use (est.)	5 / yr (est.)	n/a	~232K/yr measured	
<u>new-enemy</u> Scaffold a new enemy — enemies.json entry, BootScene placeholder sprite generator, optional test wave, and integrity test verification.	○ ESTIMATE	6K tokens / use (est.)	25 / yr (est.)	n/a	~174K/yr measured	
<u>new-migration</u> Add a Postgres schema migration end-to-end — dated SQL file in db/migrations/, Database interface update in src/lib/db.ts, prod applicati...	○ ESTIMATE	7K tokens / use (est.)	15 / yr (est.)	n/a	~61K/yr measured	
<u>new-mission</u> Scaffold a new combat mission across missions.json, waves.json, the galaxy planet binding, and a smoke test — in one shot.	○ ESTIMATE	8K tokens / use (est.)	30 / yr (est.)	n/a	~869K/yr measured	
<u>new-perk</u> Scaffold a new mission-only perk — perks.ts entry, BootScene icon generator, HUD chip wiring, and (for actives) a switch case in PerkCont...	○ ESTIMATE	9K tokens / use (est.)	10 / yr (est.)	n/a	~266K/yr measured	
<u>new-solar-system</u> Add a new solar system to the galaxy — solarSystems.json entry (incl. galaxy bed), SolarSystemId union extension, optional unlock gating,...	○ ESTIMATE	13K tokens / use (est.)	5 / yr (est.)	n/a	~29K/yr measured	
<u>new-story</u> Add, modify, or remove story content — cinematic popups, voiceovers, music beds, body text, written synopses, and auto-trigger wiring.	✓ MEASURED last 2026-04-30	56K tokens / use (measured)	1 in 90d ~4/yr projected	10× under guess was 5K/use	~55K/yr measured	
<u>new-weapon</u> Superseded by /equipment.	REDIRECT	—	—	n/a	—	
<u>save-roundtrip-audit</u> Pre-commit save-pipeline invariants check — walks every StateSnapshot field through the 8 layers of the save round-trip (snapshot interfa...	✓ MEASURED last 2026-05-03	19K tokens / use (measured)	1 in 90d ~4/yr projected	1.5× under guess was 12K/use	~29K/yr measured	

claude-skills

10 skills · 6 measured · github.com/MikkoNumminen/claude-skills

SKILL	STATUS	COST / USE		RUNS	CALIBRATION	SAVED / YR
ai-codegen-smell-audit Read-only audit for specific failure modes that recur in LLM-generated code — defensive guards on impossible cases, generic names in domain code, swallowed errors, single-use helpers, mirror tests, and so on.	✓MEASURED last 2026-05-17	199K tokens / use (measured)	1 in 90d ~4/yr projected	2.7× under guess was 75K/use	~305K/yr measured	
audit Run a multi-phase robustness audit of the current codebase.	✓MEASURED last 2026-05-20	553K tokens / use (measured)	2 in 90d ~8/yr projected	1.3× under guess was 425K/use	~8.84M/yr modeled	
mikko-audit-suite Run every `mikko-*` audit skill that's relevant to the current codebase, in a sensible order, and produce one index document linking to each audit's report.	○ESTIMATE	—	18 / yr (est.)	n/a	—	
mikko-help List every installed `mikko-*` skill with its description (or with its `barney:` field when `--barney` is passed).	✓MEASURED last 2026-05-20	644K tokens / use (measured)	2 in 90d ~8/yr projected	99× under guess was 7K/use	~10.3M/yr modeled	
mikko-install Install, update, list, or uninstall `mikko-*` skills from the cloned `claude-skills` source repo into the user-wide skill directory (`~/....`)	✓MEASURED last 2026-05-21	35K tokens / use (measured)	1 in 90d ~4/yr projected	8.8× under guess was 4K/use	~282K/yr modeled	
react-anti-patterns-audit Read-only audit for six concrete React anti-patterns that recur in component code (key=index in lists, useEffect for derived state, dep-array lies, state mutation instead of replace, missing effect cleanup, multiple sources of truth).	○ESTIMATE	23K tokens / use (est.)	30 / yr (est.)	n/a	~1.35M/yr modeled (from est.)	
readme-drift-sync Detect drift between what the repo actually contains and what `README.md` claims, then rewrite ONLY the drifted sections in the README's ...	○ESTIMATE	40K tokens / use (est.)	15 / yr (est.)	n/a	~1.20M/yr modeled (from est.)	
security-audit Orchestrates the multi-phase security audit + remediation.	○ESTIMATE	750K tokens / use (est.)	4 / yr (est.)	n/a	~17K/yr measured · costs more	

SKILL	STATUS	COST / USE		RUNS	CALIBRATION	SAVED / YR
session-cost Measure the token cost of a single Claude Code session — main thread plus every sub-agent it dispatched.	✓MEASURED last 2026-05-21	409K tokens / use (measured)	1 in 90d ~4/yr projected	<i>no prior number</i>		~3.27M/yr modeled
skill-usage Measure actual Claude Code skill usage from local transcript JSONL files.	✓MEASURED last 2026-05-21	25K tokens / use (measured)	3 in 90d ~12/yr projected	6.2× under guess was 4K/use		~597K/yr modeled

mikkonumminen.dev

4 skills · 2 measured · github.com/MikkoNumminen/mikkonumminen.dev

SKILL	STATUS	COST / USE		RUNS	CALIBRATION	SAVED / YR
md-to-pdf Render a markdown report (or any HTML) to a styled PDF using the local Chrome's `--print-to-pdf` flag.	○ESTIMATE	25K tokens / use (est.)	10 / yr (est.)	<i>n/a</i>		~500K/yr modeled (from est.)
skill-localUpdate Refresh every local artifact the site renders about your portfolio skills — the JSON the /contact terminal fetches at runtime, the measurement-overlaid annual totals, and the downloadable skills-registry PDF — in one sequenced chain.	○ESTIMATE	—	—	<i>n/a</i>		—
skill-registry Scan every sibling repo under D:/koodaamista for `.claude/skills/*/SKILL.md` files and emit a consolidated registry as a structured JSON ...	✓MEASURED last 2026-05-20	148K tokens / use (measured)	3 in 90d ~12/yr projected	1.1× under guess was 130K/use		~3.54M/yr modeled
sync-readmes Audit project data against sibling repos' READMEs and open a PR with drift corrections.	✓MEASURED last 2026-05-18	119K tokens / use (measured)	1 in 90d ~4/yr projected	1.2× over guess was 141K/use		~949K/yr modeled

Method, in plain language

How the numbers in this document are produced, what they mean, and where I am still guessing.

How “measured” rows are produced

Every time I run a Claude Code session (one conversation in the CLI from start to exit), the harness — the runtime that drives the CLI between me and the model — writes a transcript to

`~/.claude/projects/<dir>/<sessionId>.jsonl` . The file is JSON Lines: one JSON object per line, one line per message. Each assistant message in those files carries an `attributionSkill` field when a skill is active. That field is the trail.

A separate skill called `skill-usage` walks every transcript file, filters to assistant messages with an `attributionSkill` , groups by (sessionId, skillName), sums the input + output + cache-creation tokens, and dedupes by `requestId` so retried API calls do not double-count. The output is a dated `SKILL-USAGE-{YYYY-MM-DD}.json` under `.claude/agent-verdicts/` . That JSON is what the renderer reads to populate every “measured” row in this document.

Window: the last 90 days from the moment `skill-usage` ran. Sessions older than 90 days are ignored even if they're still on disk. The boundary is inclusive at the start (a session exactly 90 days old, to the second, is counted) and exclusive at the end (the very moment the scanner runs is the cutoff). Running the scanner an hour later can therefore drop a session that just crossed the boundary — that's a real if rare flake, and re-running the chain re-derives the right answer.

Annual projection: dumb linear math. If a skill ran twice in 90 days, the annual figure says about 8. That is on purpose. I would rather show the projection method honestly than dress up a single data point as a trend. When a row says *“1 in 90d → ~4/yr projected”* you should read that as *“barely a data point — trust the cost-per-use, ignore the annual.”*

Cache accounting: the per-invocation total sums `input_tokens` , `output_tokens` , and `cache_creation_input_tokens` . It does *not* sum `cache_read_input_tokens` — those are cache hits paid for upstream and roughly 10× cheaper. Counting them would double-bill a single skill's multi-turn run. If you care specifically about cache efficiency, that's a different report.

What is not counted: any skill that did not run in the 90-day window has no measured row, even if I use it often outside that window. Sub-agent token costs (work the parent skill delegates to a parallel Claude Code agent,

written to its own transcript file under a `subagents/` sibling directory) are included where the harness logs them — each sub-agent transcript inherits the parent session's `attributionSkill`. If a skill spawns work the harness does not tag, those tokens are invisible to this measurement.

Renamed skills: there's a small in-source rename map (`RENAMED_SKILLS` in `apply-measurement-overlay.mjs`) that retargets historical old-name measurements onto the renamed registry row. Without this, a rename would silently delete ~90 days of measured signal from the rendered totals. Entries get retired once the window rolls past the rename date.

How “estimated” rows are produced

I made up the number. I imagined someone — usually me — running the skill some number of times a year and costing some number of tokens per use, and I wrote down what felt right. There is no math behind these. They are guesses by the person who built the skill, written down before any measurement existed.

The estimates are still in the document because the alternative is showing only a third of the portfolio. The rows that have never been invoked in the 90-day window still represent real tools that take real tokens when used. Pretending they don't exist would make the picture cleaner and less true.

Make the asymmetry visible: every measured estimate I had has turned out to be wrong, usually low by 5× to 100×. Apply the same skepticism to the rows that have not been measured yet. If anything, the un-measured estimates are likely to be *more* wrong, because the ones I measured first were the ones I felt most confident about.

How “tokens saved” is computed

The savings column carries two kinds of numbers, marked in the rows.

Bold green numbers tagged *measured* are real A/B measurements. A Sonnet sub-agent solves a representative task in two arms — once cold (no skill awareness) and once following the skill's `SKILL.md` exactly — both in fresh sandboxed worktrees, both with token usage read from the harness's per-sub-agent `usage.total_tokens`. The saved number is arm-A tokens minus arm-B tokens for that one run. Orange numbers labelled *measured · costs more* are skills where the A/B test showed the skill arm spent more than the unstructured arm — those are real findings, not anomalies; the skill encodes rigor (e.g. a full-CRUD lifecycle or a

multi-phase audit) that the unstructured arm skipped. The arm-A / arm-B numbers are preserved on each calibrated row's receipt for downstream consumers to read.

Italic gray numbers tagged *modeled* are a 3× heuristic. When no calibration data exists for a row, the renderer assumes the unstructured alternative would cost $\sim 3\times$ a focused skill run, so saved $\approx 2\times$ cost-per-use $\times$ annual uses. The $3\times$ is a handful-of-side-by-side-runs guess from the author, not a benchmark — the May-2026 Spacepotatis calibration showed it's overstated by roughly $3\times$ at the portfolio level (measured $\sim 22\%$ rate vs the heuristic's $\sim 67\%$). Rows still showing modeled numbers are skills that haven't been A/B-tested yet; treat their savings as a possibly-too-optimistic lower bound.

Cost appears on both sides of the underlying subtraction, so the savings figure is more sensitive to bad cost estimates than the cost figure is. An italic-modeled row stacked on top of an italic-estimated cost is a model on top of a guess — least trustworthy column on the page. A bold-measured row on top of a measured cost is the most trustworthy. The visual treatment matches that hierarchy.

What this document does NOT claim

- No production cost numbers. This is development-time tooling on my own machine.
- No comparison to other engineers' setups. I only have transcripts for one engineer.
- No per-feature ROI claim. This is per-skill cost, not per-feature value.
- No assertion that any of this scales linearly to a team. It might. I have not measured.
- No claim that the modeled savings would survive an actual A/B test against well-prompted unstructured chats. They might; I haven't tested.

Reproducibility

From inside this repo (`mikkonumminen.dev/`) on a machine with Claude Code installed:

1. `/mikko-skill-usage` — writes `.claude/agent-verdicts/SKILL-USAGE-LATEST.json` from local transcripts.
2. `/skill-localUpdate` (formerly `/skill-pdf`) — re-walks the portfolio inventory, applies the measurement overlay, renders this PDF.

The data the renderer reads is committed to the repo at `public/data/skills-registry.json`. If you want to inspect what's behind a number on this page, that file is the source of truth. The dated history lives in

`.claude/agent-verdicts/SKILL-REGISTRY-*.json` .

The *last* `<date>` marker on a measured row is the timestamp of the most recent Claude Code transcript that invoked that skill. The transcript itself stays on my machine because it contains code and context from private repos. If you want to verify the measurement methodology rather than the measurement, the parser source is at

`~/claude/skills/mikko-skill-usage/SKILL.md` — it is a JSONL parser, not a network call to anything.

Generated 2026-05-22 at 15:18 UTC. Custom-skill measured cost in the 90-day window: 4.81M tokens. Modeled savings 84% measurement-backed, 16% estimate-backed. Built-in /review reference excluded from every total above.